

13 React to snaps

Next add these two lines to your code. The `bind()` function tells the GUI to listen for the “q” or “p” key being pressed, and to call the `snap()` function each time it happens. You’ll create the `snap()` function later.

```
root.after(3000, next_shape)
```

```
c.bind('q', snap)
```

```
c.bind('p', snap)
```

14 Send key presses to the GUI

The `focus_set()` function tells the key presses to go to the canvas. The GUI wouldn’t react to “q” and “p” being pressed without this function being called. Type this line below the `bind()` function calls.

```
c.bind('q', snap)
```

```
c.bind('p', snap)
```

```
c.focus_set()
```

15 Start the main loop

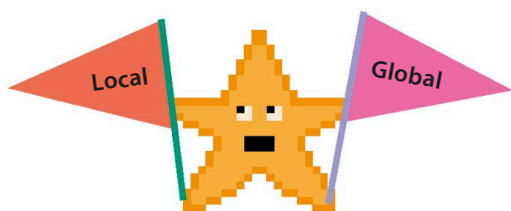
Add this line right at the end of your file. Once we add the `next_shape()` and `snap()` functions, the main loop will update the GUI with the next shape and listen for key presses.

```
c.focus_set()
```

```
root.mainloop()
```

**EXPERT TIPS****Local and global variables**

Variables can either be local or global. A local variable exists only inside a particular function, which means the rest of the program can’t use it. A variable created in the main program, outside of a function, is called global and can be used in any part of the code. However, if you want to use a function to assign a new value to a global variable, you need to add the keyword `global` before the variable’s name when you type it in the function. This is what we do in Step 16.

**Coding the functions**

The last stage is to create two functions: one to show the next shape, and another to handle snaps. Type them at the top of your program, just below the import statements.

16 Create the function

The `next_shape()` function shows the coloured shapes one after another, like cards being dealt. Start defining the function by typing the code below. It labels some of your variables as global (see box, left) and updates `previous_color`.

Using the `global` keyword ensures that changes to the variables are seen throughout the program.

```
def next_shape():
```

```
    global shape
```

```
    global previous_color
```

```
    global current_color
```

This line sets `previous_color` to `current_color` before the code gets the next shape.

```
    previous_color = current_color
```